

Lecture 17: Storage and File Structure

Amit Kumar Dhar

October 8, 2025

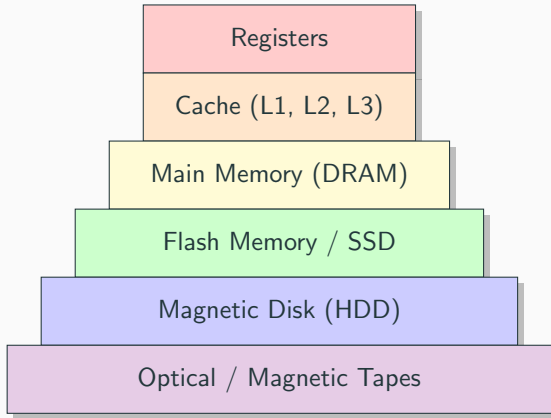
Storage Media

File Organization on Secondary Storage

Storing Records in Files

Storage Media

The Storage Hierarchy



Cost: High

Speed: Fastest

Volatility: Volatile

Primary Storage

Cost: Low

Speed: Slower

Volatility: Non-Volatile

Secondary/Tertiary Storage

Figure 1: Levels of storage, trading speed and cost for capacity and persistence.

Storage Media Types

- **Primary Storage:** Fast, volatile, and directly accessible by the CPU.
 - Cache, Main Memory (RAM)
- **Secondary Storage:** Non-volatile, moderate speed. Stores data persistently.
 - Hard Disk Drives (HDD), Solid-State Drives (SSD)
- **Tertiary Storage:** Non-volatile, very slow, high capacity. Used for backups and archives.
 - Magnetic Tapes, Optical Disks (Blu-ray)

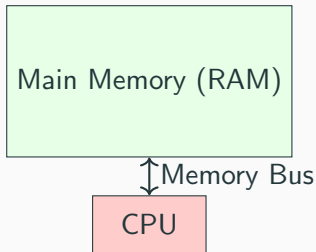
Primary Storage: Main Memory

Benefits:

- Extremely fast access times (nanoseconds).
- Directly addressable by the CPU.

Disadvantages:

- **Volatile:** Data is lost when power is turned off.
- Limited capacity due to high cost.
- More expensive per gigabyte than secondary storage.



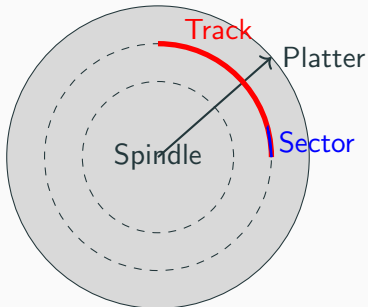
Secondary Storage: Magnetic Disk (HDD)

Benefits:

- **Non-volatile:** Data persists without power.
- Large storage capacity at a low cost.
- Good for sequential access.

Disadvantages:

- Slow random access times (milliseconds) due to mechanical parts (seek time, rotational latency).
- Prone to physical damage.



Anatomy of a Disk Read/Write

To optimize access, we must understand what makes it slow. Total time is the sum of three components:

- **Seek Time:** Time for the arm to move the heads to the correct cylinder. **This is the most significant delay.**
- **Rotational Latency:** Time for the desired sector to rotate under the read/write head. On average, this is half a rotation.
- **Transfer Time:** Time to actually read the data from the platter as it spins past the head. This is typically the fastest part.

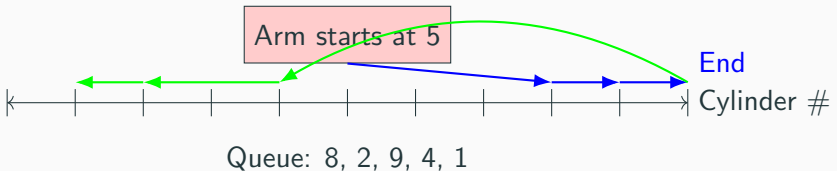
Optimization Goal

The primary goal of disk optimization algorithms is to **minimize total seek time**.

Disk Scheduling: The Elevator (SCAN) Algorithm

The Operating System can reorder pending disk I/O requests to improve efficiency.

- The disk arm moves in one direction (e.g., towards increasing cylinder numbers), servicing all requests in its path.
- When it reaches the end, it reverses direction and services requests on its way back.
- This is much more efficient than First-Come, First-Served (FCFS), which can cause wild arm movements.



Path: 5 → 8 → 9 → 10 (turn) → 4 → 2 → 1

Optimizing Data Placement

How data is physically laid out on the disk is critical.

- **Data Clustering (Aligning):** Store related data blocks as close together as possible (e.g., on the same track or cylinder).
 - This turns multiple slow, random seeks into a single seek followed by a fast, sequential read.
 - Example: A database might store all records for a given table in contiguous blocks.
- **Swap Space:**
 - A dedicated partition on the disk used for virtual memory.
 - The OS moves inactive memory pages to this space.
 - While essential, heavy use of swapping ("thrashing") causes constant disk I/O and can severely degrade system performance.

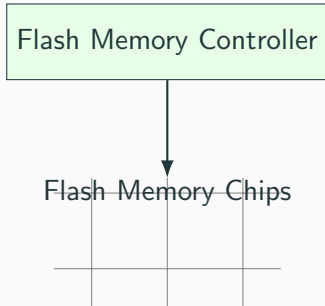
Secondary Storage: Solid-State Drive (SSD)

Benefits:

- **Non-volatile.**
- Much faster than HDDs, especially for random access (no moving parts).
- More durable and power-efficient.

Disadvantages:

- More expensive per gigabyte than HDDs.
- Limited number of write cycles (though this is less of a concern for modern SSDs).



SSD Write Endurance Problem

- SSDs are made of NAND flash memory cells.
- Each cell can only be written to a limited number of times before it wears out and becomes unreliable. This is called **write endurance**.
- If the file system repeatedly wrote to the same blocks (e.g., for a file allocation table or a journal), those blocks would fail very quickly, rendering the drive useless.
- **Challenge:** How to distribute writes evenly across all cells to maximize the drive's lifespan?

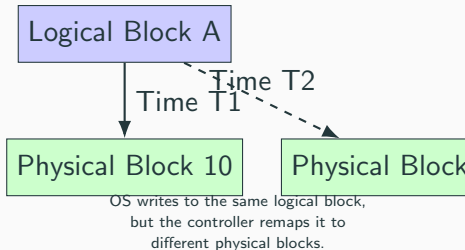
Solution

The SSD's internal controller uses sophisticated techniques to manage this, primarily **wear leveling**.

Techniques to Mitigate SSD Wear

Wear Leveling

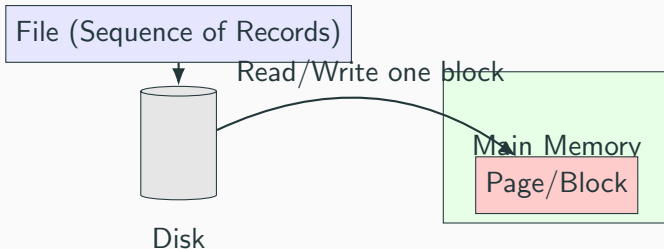
- The SSD controller ensures that write operations are distributed evenly across all physical blocks of the drive.
- It maintains a map of logical block addresses (from the OS) to physical block addresses.
- When data is overwritten, the controller writes the new data to a different physical block and updates the map.
- This prevents any single block from being written to



File Organization on Secondary Storage

Blocks and Pages

- Data is transferred between disk and main memory in units called **blocks**.
- A block is a contiguous sequence of bytes and is the smallest unit of data transfer.
- The size of a block is fixed, typically ranging from 4KB to 16KB.
- For a DBMS, a block is often referred to as a **page**.
- Records are stored within these pages.



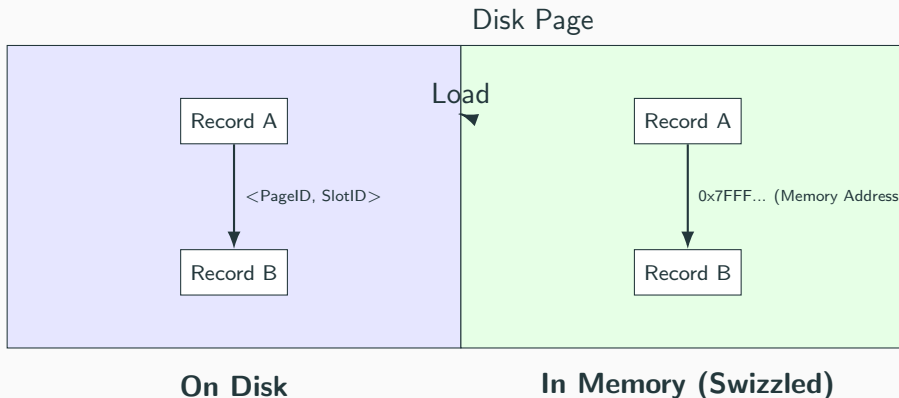
Tricks to Make Access Faster

How can we reduce the time it takes to get data from disk?

- **Buffering:** Keep frequently used blocks in main memory.
(More on this later!)
- **File Organization:** Arrange records intelligently on disk to minimize block transfers.
- **Indexing:** Create data structures (like B+-trees) to quickly find the blocks containing desired records.
- **Pointer Swizzling:** Convert disk pointers to memory pointers for faster access once data is in memory.

Pointer Swizzling

- **On-disk pointers** are logical (e.g., `<page_id, slot_id>`).
- When a page is loaded into memory, we can "swizzle" these pointers into direct **in-memory pointers** (memory addresses).
- This avoids repeated lookups in the buffer manager.



Storing Records in Files

Storing Records in a File

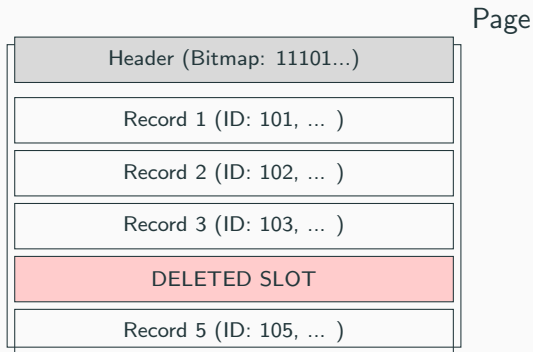
- A file is a collection of pages.
- Each page contains a collection of records.
- A **record** is a group of related data fields.
- How we store records within a page depends on whether they are fixed-length or variable-length.

Example Record: Student

```
Student(ID: int, Name:  string, Dept:  string,  
Credits:  int)
```

Storing Fixed-Length Records

- All records have the same size.
- Storage is simple: divide the page into slots of the record size.
- **Finding record i :** $\text{Location} = \text{start} + (i - 1) * \text{record_size}$.
- **Deletion:** Can leave empty slots. Need a way to manage free space (e.g., a free list or bitmap in the page header).



Handling Deletion and Insertion (Fixed-Length)

When a record is deleted, we have two main options to manage the empty slot:

Option 1: Free List

- Store a pointer to the first deleted slot in the page header.
- Each deleted slot stores a pointer to the next deleted slot.
- **Insertion:** Use a slot from the free list.
- **Deletion:** Add the newly empty slot to the front of the free list.

Option 2: Move Records

- When a record is deleted, shift subsequent records to fill the gap.
- This keeps all records contiguous.
- **Problem:** This is expensive and invalidates record IDs (RIDs) that are based on slot numbers.
- Generally not preferred if RIDs need to be stable.

A bitmap in the header (as shown previously) is another way to

Storing Variable-Length Records

Records do not have the same size (e.g., due to variable-length strings).

- **Challenge:** How to store records compactly and find them efficiently?
- **Method 1: Byte-String Representation**
 - Store records one after another, separated by a special end-of-record symbol (#).
 - Simple, but inefficient for finding the Nth record (requires scanning).
- **Method 2: Slotted Page Structure**
 - The preferred method in most DBMS.

Variable-Length: Byte-String Representation

- This is one of the simplest methods for storing variable-length records.
- Records are stored contiguously, one after the other.
- A special character or byte sequence (delimiter) is used to mark the end of each record.
- Null values can be represented by two consecutive delimiters.

Example Record: Student

`Student(ID: int, Name: string, Dept: string)`

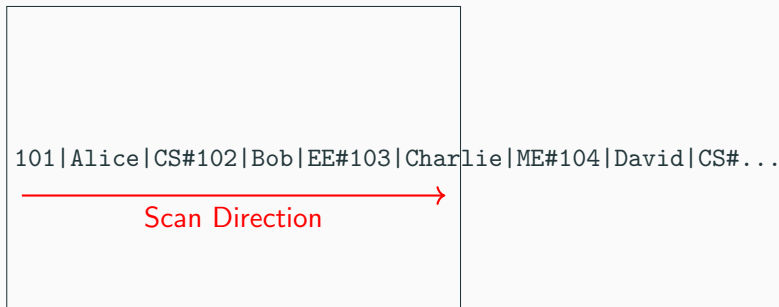
- Record 1: `101|Alice|CS#`
- Record 2: `102|Bob|EE#`
- Record 3: `103|Charlie|ME#`

Here, `'|'` separates fields and `'#'` separates records.

Byte-String Representation: Illustration

- The page is essentially treated as a long sequence of bytes.
- To find a record, the page must be scanned from the beginning, looking for the end-of-record delimiters.

Page



Byte-String Representation: Analysis

Benefits:

- **Simple to implement:** No complex page structure or headers are needed.
- **Space efficient:** No wasted space for pointers or slot directories, especially if records are small.

Disadvantages:

- **Slow Searches:** Finding the Nth record requires scanning and parsing the first N-1 records.
- **Difficult Updates:** If a record grows, it may need to be moved, which requires shifting all subsequent data.
- **Deletion is complex:** Deleting a record leaves a gap. This space can be hard to reuse, often requiring page

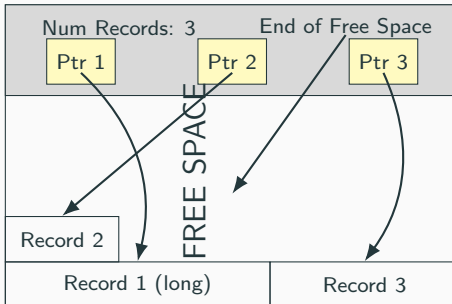
Variable-Length: Slotted Page Structure

A header at the beginning of the page contains:

- Number of record entries.
- Pointer to the end of free space within the page.
- An array of <location, size> entries for each record.

Records are allocated from the end of the page.

Slotted Page



Benefits of Slotted Page

- Records can be moved within the page without changing external pointers (only the slot directory pointer needs updating).
- Supports variable-length records efficiently.
- Deletion is handled by setting the slot pointer to null and reclaiming space.

Questions?